JavaScript — Zero to Master

25 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (7 topics)

01 Introduction and Setup
02 Variables and Data Types
03 Operators and Type Coercion
04 Conditions and Loops
05 Functions
06 Arrays and Objects
07 DOM Basics

Intermediate (7 topics)

08 Array Methods (map, filter, reduce)
09 Scope, Hoisting and Closures
10 this, call, apply, bind
11 Objects, Prototypes and Classes
12 ES Modules
13 Error Handling
14 JSON, Storage and Dates

Advanced (7 topics)

15 Asynchronous JavaScript: Callbacks and Promises
16 async / await and fetch
17 Event Loop, Microtasks and Timers
18 Iterators, Generators, Symbols
19 Higher-Order Functions and Functional Style
20 Proxies, Reflect, WeakMap

21	Browser APIs

Master (4 topics)

22	Performance and Memory
23	Design Patterns and Clean Code
24	Testing and Tooling
25	Security Basics

Beginner Level

01. Introduction and Setup

JavaScript is the programming language of the web; it runs in browsers and on servers (Node.js). Run code in the browser console, a `<script>` tag, or with `node file.js`.

```
console.log('Hello, JavaScript!');
```

02. Variables and Data Types

Use `const` by default and `let` when a value changes; avoid `var`. Types: string, number, boolean, null, undefined, bigint, symbol, object.

```
const name = 'Arun';
let age = 25;
console.log(typeof name, typeof age);
```

03. Operators and Type Coercion

Use `===` (strict equality) instead of `==`. Falsy values: `false, 0, '', null, undefined, NaN`. Use `??` for defaults and `?.` for safe access.

```
console.log(0 == '0', 0 === '0'); // true false
const city = user?.address?.city ?? 'Unknown';
```

04. Conditions and Loops

`if/else`, `switch`, `for`, `while`, `for...of` (values) and `for...in` (keys).

```
for (const n of [1, 2, 3]) {
  if (n % 2) console.log(n, 'odd');
}
```

05. Functions

Declare with `function`, function expressions or arrow functions. Support default and rest parameters; functions are values and can be passed around.

```
const add = (a, b = 0) => a + b;
const sum = (...nums) => nums.reduce((t, n) => t + n, 0);
```

06. Arrays and Objects

Arrays hold ordered lists (`push`, `pop`, `slice`); objects hold key-value pairs. Use destructuring and the spread operator to copy and unpack.

```
const [a, b] = [10, 20];
const { name, ...rest } = { name: 'Priya', age: 22, city: 'Madurai' };
const copy = { ...rest, age: 23 };
```

07. DOM Basics

The DOM is the page as objects. Select elements with `querySelector`, change `textContent`/classes and react to events with `addEventListener`.

```
document.querySelector('#btn').addEventListener('click', () => {
  document.querySelector('#msg').textContent = 'Clicked!';
});
```

Intermediate Level

08. Array Methods (map, filter, reduce)

Transform data without loops: `map` (transform), `filter` (keep), `reduce` (combine), `find`, `some`, `every`, `sort`.

```
const total = [10, 20, 30].filter(n => n > 10).map(n => n * 2).reduce((t, n) => t + n, 0); //
100
```

09. Scope, Hoisting and Closures

`let` / `const` are block scoped. A **closure** is a function that remembers variables from where it was created, enabling private state.

```
function counter() {
  let c = 0;
  return () => ++c;
}
const next = counter(); next(); next(); // 2
```

10. this, call, apply, bind

`this` depends on how a function is called. Arrow functions inherit `this` from outside. `bind` fixes `this` permanently.

```
const user = { name: 'Ravi', hi() { return 'Hi ' + this.name; } };
const f = user.hi.bind({ name: 'Meena' });
```

11. Objects, Prototypes and Classes

Objects inherit through the prototype chain. `class` syntax is a cleaner way to write constructors, methods, `static` members and `extends`.

```
class Animal {
  constructor(name) { this.name = name; }
  speak() { return `${this.name} makes a sound`; }
}
class Dog extends Animal { speak() { return `${this.name} barks`; } }
```

12. ES Modules

Split code into files with `export` and `import`; modules have their own scope and are loaded once.

```
// math.js
export const add = (a, b) => a + b;
// app.js
import { add } from './math.js';
```

13. Error Handling

Use `try/catch/finally` and throw `Error` objects. Always handle failures in asynchronous code.

```
try { JSON.parse('{bad json'); }
catch (e) { console.error('Invalid JSON:', e.message); }
```

14. JSON, Storage and Dates

`JSON.stringify/parse` convert data to text; `localStorage` stores strings in the browser; `Date` and `Intl` format dates and numbers.

```
localStorage.setItem('user', JSON.stringify({ id: 1 }));
const u = JSON.parse(localStorage.getItem('user'));
```

Advanced Level

15. Asynchronous JavaScript: Callbacks and Promises

Async work (network, timers) finishes later. A **Promise** represents a future value with states pending/fulfilled/rejected and is chained with `then/catch`.

```
fetch('/api/users')
  .then(r => r.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

16. async / await and fetch

`async/await` makes promises read like normal code. Use `Promise.all` to run tasks in parallel.

```
async function load() {
  const [users, posts] = await Promise.all([
    fetch('/users').then(r => r.json()),
    fetch('/posts').then(r => r.json()),
  ]);
  return { users, posts };
}
```

17. Event Loop, Microtasks and Timers

JavaScript is single threaded. The event loop runs the call stack, then **microtasks** (promises), then **macrotasks** (timers). This explains output order.

```
console.log(1);
setTimeout(() => console.log(2));
Promise.resolve().then(() => console.log(3));
console.log(4); // 1 4 3 2
```

18. Iterators, Generators, Symbols

Objects with `[Symbol.iterator]` work in `for...of`. **Generators** (`function*`) pause and resume with `yield`.

```
function* ids() { let i = 1; while (true) yield i++; }
const g = ids(); g.next().value; // 1
```

19. Higher-Order Functions and Functional Style

Functions that take or return functions: currying, composition, pure functions and immutability make code predictable and testable.

```
const compose = (...fns) => x => fns.reduceRight((v, f) => f(v), x);
const add = a => b => a + b;
```

20. Proxies, Reflect, WeakMap

`Proxy` intercepts object operations (validation, reactivity); `WeakMap` stores private data without preventing garbage collection.

```
const p = new Proxy({}, { set(o, k, v) { if (k === 'age' && v < 0) throw Error('bad'); o[k] =
v; return true; } });
```

21. Browser APIs

Fetch, `IntersectionObserver`, Web Storage, `History`, Web Workers, WebSockets, Canvas and the Clipboard API power rich apps.

```
new IntersectionObserver(([e]) => e.isIntersecting && console.log('visible')).observe(el);
```

Master Level

22. Performance and Memory

Avoid layout thrashing, debounce events, use `requestAnimationFrame`, lazy load code, and avoid memory leaks (forgotten listeners, timers, big closures).

```
const debounce = (fn, ms) => { let t; return (...a) => { clearTimeout(t); t = setTimeout(() =>
fn(...a), ms); }; };
```

23. Design Patterns and Clean Code

Module, Observer, Factory, Singleton and Strategy patterns; keep functions small, name clearly, and avoid global state.

```
class Emitter {
  #h = {};
  on(e, f) { (this.#h[e] ??= []).push(f); }
  emit(e, d) { (this.#h[e] || []).forEach(f => f(d)); }
}
```

24. Testing and Tooling

Test with Vitest/Jest, lint with ESLint, format with Prettier, bundle with Vite; use npm scripts and Git hooks.

```
npm create vite@latest
npm i -D vitest eslint prettier
```

25. Security Basics

Prevent XSS (never insert untrusted HTML), use HTTPS, validate input, avoid `eval`, and keep dependencies updated (`npm audit`).

```
el.textContent = userInput; // safe
// el.innerHTML = userInput; // dangerous
```

Interview Question Bank

Q1. var vs let vs const?
`var` is function scoped and hoisted; `let` and `const` are block scoped; `const` cannot be reassigned.

Q2. What is a closure?
A function that keeps access to variables of its outer scope after the outer function has returned.

Q3. == vs ===?
`==` coerces types; `===` compares value and type without coercion.

Q4. Explain the event loop.
The call stack runs synchronous code; completed async callbacks wait in queues; microtasks (promises) run before macrotasks (timers).

Q5. What is `this`?
The object a function is called on; arrow functions use the surrounding `this`.

Q6. Promise vs async/await?
`async/await` is syntax sugar over promises for cleaner sequential code.

Q7. What is hoisting?
Declarations are processed before execution; `var` and functions are hoisted, `let/const` stay in the temporal dead zone.

Q8. Shallow vs deep copy?
Spread/`Object.assign` copy one level; use `structuredClone` for deep copies.

Q9. What is debouncing vs throttling?
Debounce waits until activity stops; throttle runs at most once per interval.

Q10. Prototype vs class?
Classes are syntax over prototypal inheritance.

Capstone Project Ideas

1. To-do app with localStorage
2. Weather app using a public API
3. Quiz game with timer
4. Expense tracker with charts
5. Markdown note-taking app